IV. LESSON PROPER

Introduction to SQL Injection
SQL injection (SQLi) attacks target web applications that use SQL databases. They exploit vulnerabilities in the way SQL queries are constructed, allowing attackers to manipulate the database behind the application. Given the widespread use of SQL databases, SQL injection poses significant risks to web security.

The purpose of SQL injection attacks varies, but common objectives include:

1. **Data Retrieval**: Attackers may attempt to extract sensitive information such as usernames, passwords, credit card numbers, or personal data stored within the database.
2. **Data Manipulation**: Attackers may alter, add, or delete data within the database, potentially causing data corruption or disruption of application functionality.
3. **Authentication Bypass**: By injecting SQL code, attackers may attempt to bypass login authentication mechanisms, gaining unauthorized access to restricted areas of the application.
4. **Application Defacement**: SQL injection can also be used to deface websites by modifying content or executing arbitrary code on the server.

Impact on Web Applications:
The impact of SQL injection attacks on web applications can be severe and wide-ranging:

1. **Data Breach**: SQL injection can lead to the exposure of sensitive information stored in databases, compromising the confidentiality and privacy of users' data. This can result in legal consequences, financial losses, and damage to the organization's reputation.
2. **Data Loss or Corruption**: Attackers can manipulate SQL queries to delete or modify critical data, leading to data loss, integrity violations, and disruption of business operations.
3. **Unauthorized Access**: SQL injection can allow attackers to bypass authentication mechanisms and gain unauthorized access to administrative features or sensitive areas of the application, enabling further exploitation or malicious activities.
4. **Denial of Service (DoS)**: In some cases, SQL injection attacks can be used to execute resource-intensive queries, consuming server resources and causing performance degradation or even server downtime, leading to service disruptions for legitimate users.
5. **Compromised User Experience**: Web applications affected by SQL injection may exhibit unexpected behaviors, errors, or anomalies, resulting in a degraded user experience and loss of trust among users.

Types of SQL Injection:

1. **Union-Based SQL Injection:** Union-based SQL injection exploits the ability to combine the results of two or more SQL queries into a single result set using the UNION operator. Attackers leverage this technique to inject additional SQL code that retrieves data from other database tables or performs arbitrary queries. By carefully crafting the injected SQL code, attackers can extract sensitive information or manipulate the application's behavior.
2. **Blind SQL Injection:** Blind SQL injection occurs when an attacker cannot directly view the results of their injected SQL queries but can infer information through the application's responses. This type of injection relies on boolean-based or time-based techniques to extract data or infer database structure by sending specially crafted SQL payloads and analyzing the application's responses for true/false or delay indicators. Blind SQL injection is often used when direct extraction of data is not feasible, such as when error messages

are suppressed.

3. **Error-Based SQL Injection:** Error-based SQL injection exploits error messages generated by the database server when executing malformed SQL queries. Attackers intentionally inject erroneous SQL code into input fields, causing the database server to produce error messages containing valuable information about the underlying database structure or data. By analyzing these error messages, attackers can gather intelligence and refine their exploitation strategy, potentially leading to data exfiltration or further compromise of the application.

What Is SQL?

Structured Query Language (SQL) is used for managing and manipulating relational databases. In a relational database:

- **Data is stored in tables**: Each table consists of rows (records) and columns (fields).
- **Basic SQL operations**:
  - **INSERT**: Adds new records.
  - **SELECT**: Retrieves data.
  - **UPDATE**: Modifies existing records.
  - **DELETE**: Removes records.

Anatomy of a SQL Injection Attack

SQL injection occurs when a web application constructs SQL queries using unsanitized user input. This can lead to unauthorized database access and manipulation.

Example of Insecure SQL Construction

Consider the following insecure Java code snippet used for user authentication:

*String query = "SELECT * FROM users WHERE email = '" + email + "' AND password = '" + password + "'";*

If an attacker inputs billy@gmail.com'-- as the email, the constructed SQL query becomes:

*SELECT * FROM users WHERE email = 'billy@gmail.com'--' AND password = '';*

Here, the -- indicates a comment, effectively bypassing password validation and allowing the attacker to log in without a password.

Advanced SQL Injection

A more complex attack could involve executing additional commands, such as:

billy@gmail.com'; DROP TABLE users;--

This would result in two SQL commands being executed:

1. Retrieve user data (which may succeed).
2. Drop the users table, leading to data loss.

Consequences of SQL Injection

If successful, SQL injection can allow attackers to:

- Bypass authentication mechanisms.
- Read, modify, or delete data.
- Execute administrative operations on the database.
- Inject malicious scripts that compromise user security.

Prevention and Mitigation:

1. Prepared Statements and Parameterized Queries: Prepared statements and parameterized queries are SQL execution methods that separate SQL code from user input. Instead of directly concatenating user-supplied values into SQL queries, placeholders are used to represent parameters. These placeholders are then bound to user input values at runtime. This approach prevents attackers from injecting malicious SQL code into queries, as the database treats user input strictly as data rather than executable code. Prepared statements and parameterized queries are supported by most modern programming languages and database APIs and are highly effective in preventing SQL injection attacks.
2. Input Validation and Sanitization: Input validation involves inspecting and filtering user-supplied data to ensure it conforms to expected formats, lengths, and character sets. Sanitization techniques remove or escape potentially harmful characters from input strings before they are used in SQL queries. While input validation and sanitization alone are not sufficient to prevent SQL injection attacks, they serve as an additional layer of defense by reducing the attack surface and limiting the impact of malicious input. However, it's essential to note that input validation should be used in conjunction with other security measures, as it may not catch all forms of malicious input.
3. Using ORM (Object-Relational Mapping) Frameworks: ORM frameworks abstract database interactions by mapping database objects to application objects, reducing the need for explicit SQL queries. ORM frameworks handle SQL query generation and parameter binding internally, reducing the likelihood of SQL injection vulnerabilities. Additionally, ORM frameworks often provide built-in protection against common security threats, such as SQL injection and cross-site scripting (XSS), through features like query parameterization, input validation, and escaping mechanisms. However, it's crucial to use ORM frameworks securely and keep them up to date with the latest security patches to mitigate potential risks.

Mitigation Strategies

1. Use Parameterized Statements

To safeguard against SQL injection:

- Construct SQL queries using bind parameters, which act as placeholders that the database driver securely replaces with user inputs.

Example in Java

Using PreparedStatement:

```
PreparedStatement pstmt = connection.prepareStatement("SELECT * FROM users
WHERE email = ? AND password = ?");
pstmt.setString(1, email);
pstmt.setString(2, password);
```

2. Utilize Object-Relational Mapping (ORM)

ORM libraries abstract SQL construction and allow developers to interact with data using objects. While ORMs use parameterized queries, be cautious when using raw SQL queries.

3. Defense in Depth

Implement multiple layers of security:

- Always sanitize and validate user inputs.

- Implement Web Application Firewalls (WAFs).
- Regularly audit your application for vulnerabilities.

4. Principle of Least Privilege

Limit database access permissions:

- Use accounts with minimal privileges necessary for web applications.
- Restrict accounts from executing dangerous SQL statements (e.g., CREATE, DROP).

Types of SQL Injection Attacks

Blind vs. Nonblind SQL Injection

- **Nonblind SQL Injection**: Attacker receives detailed error messages from the database, making it easier to exploit.
- **Blind SQL Injection**: Attacker infers data by observing application behavior rather than receiving error details.

Command Injection

This occurs when an attacker can execute arbitrary commands on the underlying operating system through insecure command execution paths.

Mitigation for Command Injection

1. **Escape Control Characters**: Ensure user inputs do not contain harmful control characters.
2. **Input Validation**: Rigorously check and sanitize inputs.

Remote Code Execution (RCE)

RCE attacks occur when attackers exploit vulnerabilities in web applications to execute arbitrary code.

Mitigation

- **Disable Code Execution During Deserialization**: Avoid insecure deserialization practices that allow execution of untrusted code.

File Upload Vulnerabilities

File uploads can be exploited if not properly secured, allowing attackers to upload malicious scripts (web shells) that could compromise the server.

Mitigations for File Uploads

1. **Validate File Types**: Restrict uploads to only allowed file types (e.g., images).
2. **Use Anti-Virus Scanning**: Scan uploaded files for malware.
3. **Limit Upload Size**: Set size limits to mitigate potential abuse.

Remote Code Execution (RCE):

Understanding Remote Code Execution:

Definition and Significance: Remote Code Execution (RCE) is a critical cybersecurity vulnerability and attack vector that allows attackers to execute arbitrary code on a target system or application remotely, without requiring physical access or direct user interaction. This type of attack typically occurs when a system or application fails to properly validate or sanitize input data, allowing attackers to inject and execute malicious code from a remote location.

RCE vulnerabilities are highly significant because they provide attackers with full control over

the compromised system or application. By exploiting RCE vulnerabilities, attackers can execute commands, install malware, modify configurations, exfiltrate sensitive data, or even take over entire systems, posing severe risks to confidentiality, integrity, and availability.

Consequences of RCE Attacks:

1. **Complete System Compromise:** RCE attacks enable attackers to execute arbitrary code with the same privileges as the vulnerable application or service. This can lead to complete system compromise, allowing attackers to gain unauthorized access, escalate privileges, and take full control over the compromised system. Once compromised, attackers can install backdoors, rootkits, or persistent malware, enabling them to maintain access and control even after remediation attempts.
2. **Data Theft and Exfiltration:** Attackers can leverage RCE vulnerabilities to access and exfiltrate sensitive data stored on the compromised system. This includes personally identifiable information (PII), financial records, intellectual property, credentials, and other confidential information. Data theft and exfiltration can result in financial losses, regulatory penalties, legal liabilities, and reputational damage for affected organizations.
3. **Malware Deployment and Propagation:** RCE vulnerabilities provide attackers with the ability to deploy and execute malicious payloads on the compromised system, including viruses, worms, ransomware, and other types of malware. Once deployed, malware can propagate across networks, infecting other systems and devices, and causing widespread disruption and damage. RCE-based malware attacks can lead to service disruptions, data corruption, and financial losses for targeted organizations.
4. **System Downtime and Service Disruption:** RCE attacks can result in system downtime and service disruption as attackers exploit vulnerable systems to execute malicious code, launch denial-of-service (DoS) attacks, or perform unauthorized actions. Service disruptions can impact business operations, customer services, and critical infrastructure, leading to financial losses, productivity setbacks, and damage to organizational reputation.
5. **Regulatory Compliance Violations:** Organizations that fall victim to RCE attacks may face regulatory compliance violations, particularly in industries governed by strict data protection regulations such as GDPR, HIPAA, PCI DSS, or SOX. Failure to protect sensitive data, prevent unauthorized access, or respond effectively to security incidents can result in regulatory fines, legal sanctions, and reputational damage, further exacerbating the consequences of RCE attacks.

Exploitation Techniques:

1. **Injecting and Executing Malicious Code on a Remote Server:** Remote Code Execution (RCE) vulnerabilities allow attackers to inject and execute arbitrary code on a target server or application remotely. Attackers exploit these vulnerabilities by sending specially crafted input data, such as HTTP requests, SQL queries, or command-line parameters, containing malicious code payloads. When the vulnerable server processes the malicious input without proper validation or sanitization, it inadvertently executes the injected code within its execution environment.

Exploiting RCE vulnerabilities typically involves the following steps:

- Identifying and exploiting the RCE vulnerability: Attackers identify and exploit RCE vulnerabilities in web applications, network services, or software components by sending payloads containing malicious code snippets, command injection payloads, or serialized objects that trigger code execution.
- Crafting and delivering the exploit payload: Attackers craft exploit payloads containing shell commands, scripting languages (e.g., PHP, Python, or Perl), operating system commands, or binary payloads designed to achieve specific

objectives, such as gaining unauthorized access, extracting sensitive information, or installing malware.
    - Leveraging insecure deserialization: In some cases, attackers exploit insecure deserialization vulnerabilities to execute arbitrary code by manipulating serialized objects or data structures sent to the target server. By crafting malicious payloads that exploit flaws in deserialization routines, attackers can execute code remotely and achieve various malicious goals.
2. **Gaining Unauthorized Access to the Server:** RCE vulnerabilities enable attackers to gain unauthorized access to the target server or application by executing arbitrary code within its execution context. Once attackers successfully exploit an RCE vulnerability, they can execute commands with the same privileges as the vulnerable process or application, potentially leading to full system compromise.

Exploiting RCE vulnerabilities to gain unauthorized access typically involves the following techniques:

- Privilege escalation: Attackers exploit RCE vulnerabilities to escalate their privileges and gain administrative access to the target server or application. By executing commands with elevated privileges, attackers can bypass access controls, modify system configurations, or install persistent backdoors.
- Post-exploitation activities: After gaining unauthorized access, attackers perform post-exploitation activities, such as reconnaissance, lateral movement, data exfiltration, or further exploitation of internal resources. Attackers may use compromised servers as footholds to launch additional attacks, pivot within the network, or maintain persistence for future exploitation.

Prevention and Mitigation:

1. **Principle of Least Privilege:** Adhering to the principle of least privilege is crucial for mitigating the risk of Remote Code Execution (RCE) vulnerabilities. Organizations should restrict user and application privileges to the minimum level necessary to perform their intended functions. By granting only the privileges required for specific tasks, organizations can limit the potential impact of RCE attacks. For example, web servers, database services, and other network-facing applications should run with minimal privileges to prevent attackers from exploiting RCE vulnerabilities to escalate privileges and gain unauthorized access to critical system resources.
2. **Secure Coding Practices (e.g., Input Validation, Output Encoding):** Secure coding practices play a fundamental role in preventing RCE vulnerabilities by reducing the attack surface and enforcing robust security controls throughout the software development lifecycle. Developers should implement the following practices:
    - Input validation: Validate and sanitize all user-supplied input to ensure it conforms to expected formats, lengths, and data types. Reject input that contains potentially malicious characters or patterns to prevent injection attacks, including code injection, SQL injection, and command injection.
    - Output encoding: Encode output data to prevent cross-site scripting (XSS) attacks, which can be leveraged to inject and execute malicious JavaScript code in users' browsers. Use appropriate encoding techniques, such as HTML entity encoding or JavaScript escaping, to neutralize malicious input and prevent client-side script execution.
    - Secure development frameworks: Use secure development frameworks and libraries that provide built-in protections against common security vulnerabilities, including RCE vulnerabilities. Leveraging well-maintained, security-reviewed libraries and frameworks can help reduce the likelihood of introducing RCE vulnerabilities into software applications.

3. **Regular Security Audits and Vulnerability Assessments:** Conducting regular security audits and vulnerability assessments is essential for identifying and remediating RCE vulnerabilities in software applications, network services, and infrastructure components. Organizations should perform comprehensive security assessments, penetration tests, and code reviews to identify potential security weaknesses, including RCE vulnerabilities. By proactively identifying and addressing security vulnerabilities, organizations can reduce the risk of exploitation by malicious actors and strengthen their overall security posture.

Security audits and vulnerability assessments should encompass the following activities:

- Vulnerability scanning: Use automated vulnerability scanning tools to identify known security vulnerabilities, misconfigurations, and weak points in software applications and network infrastructure.
- Penetration testing: Conduct controlled security assessments, penetration tests, and red team exercises to simulate real-world attack scenarios and assess the effectiveness of existing security controls in detecting and mitigating RCE vulnerabilities.
- Code review: Perform manual and automated code reviews to analyze source code for potential security vulnerabilities, including input validation flaws, insecure coding practices, and code injection vulnerabilities. Ensure that developers follow secure coding guidelines and best practices throughout the software development process.